Solution: Majority Element

Problem Statement

Given an array `nums` having an `n` elements, identify the element that appears the majority of the time, meaning more than n/2 times.

Examples

1. **Example 1**:
   - Input: `[1, 2, 2, 3, 2]`
   - Expected Output: `2`
   - Justification: Here, '2' appears 3 times in a 5-element array, making it the majority element.
2. **Example 2**:
   - Input: `[4, 4, 4, 4, 7, 4, 4]`
   - Expected Output: `4`
   - Justification: '4' is the majority element as it appears 5 out of 7 times.
3. **Example 3**:
   - Input: `[9, 9, 1, 1, 9, 1, 9, 9]`
   - Expected Output: `9`
   - Justification: '9' is the majority element, appearing 5 times in an 8-element array.

Solution

The divide and conquer algorithm can be used to find the majority element in an array efficiently. The strategy involves recursively dividing the array into two halves until each segment contains a single element. At this base level, a single element is trivially the majority of its one-element segment. As we merge these segments back together, we determine the majority element of each merged segment by comparing the counts of the majority elements from the two halves. The majority element of the entire array is the one that remains the majority as segments are progressively merged.

Here is the step-by-step algorithm.

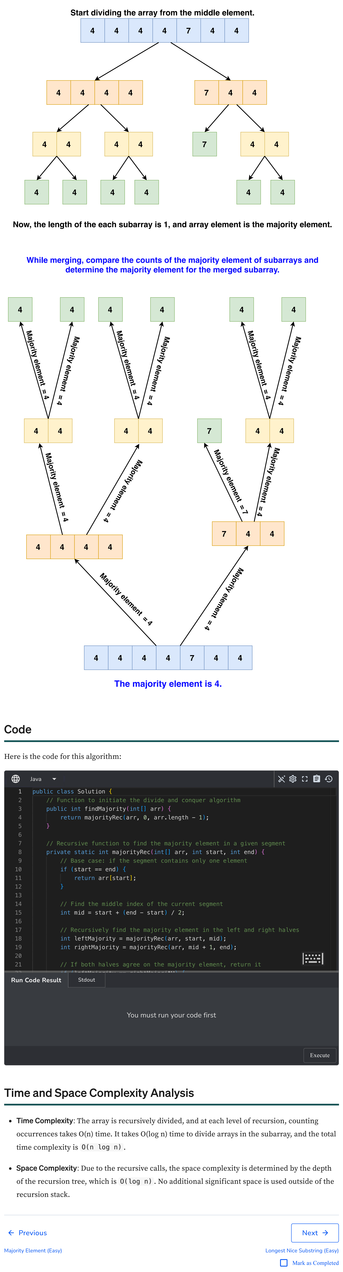
1. **Base Case:** In the recursive function, check if the current segment of the array has only one element. If so, return this element as it is trivially the majority element of this single-element segment.
2. **Divide Step:**
   - Calculate the middle index of the current segment of the array.
   - Recursively call the function for the left half of the array (`start to middle`) and the right half (`middle + 1 to end`).
   - Store the returned values from both halves, which are potential majority elements for each half.
3. **Conquer Step:**
   - If the majority element found in the left half is the same as the one in the right half, return this element as the majority element for the current segment.
   - If they are different, count the occurrences of each within the current segment.
4. **Combine Step:**
   - Compare the counts of these two elements.
   - Return the element that has a higher count within the current segment. If the counts are equal, either can be returned as the majority element for this segment.
5. **End Function:** Return the majority element of the array.

This logic is based on the principle that the majority element of the entire array must be the majority element in at least one of the two halves. By recursively breaking down and then combining the array, we are able to zero in on the majority element efficiently.

Algorithm Walkthrough

1. **Initial Array**: `[4, 4, 4, 4, 7, 4, 4]`
2. **Divide**: Split into `[4, 4, 4, 4]` and `[7, 4, 4]`.
   - For `[4, 4, 4, 4]`, split further into `[4, 4]` and `[4, 4]`.
     - `[4, 4]` splits into `[4]` and `[4]`, both trivially `4`.
     - Merge `[4]` and `[4]` to get `4` as the majority.
     - Merge `[4, 4]` and `[4, 4]` to get `4` as the majority.
   - For `[7, 4, 4]`, split into `[7]` and `[4, 4]`.
     - `[7]` is trivially their own majority.
     - Merge `[4]` and `[7]`, neither is a majority, so no majority for this segment.
     - `[4, 4]` splits into `[4]` and `[4]`, both trivially `4`.
     - Merge `[4]` and `[4]` to get `4` as the majority.
     - Merge `[7]` and `[4, 4]`, `4` is the majority.
3. **Combine**: Merge `[4, 4, 4, 4]` and `[7, 4, 4]`.
   - Count occurrences of `4` in the entire array (5 times).
   - Since 5 is more than half of occurences of `7`, `4` is the majority of the whole array.

Here is the visual representation of the algorithm:



Code

Here is the code for this algorithm:

```java
public class Solution {
    // Function to initiate the divide and conquer algorithm
    public int findMajority(int[] arr) {
        return majorityRec(arr, 0, arr.length - 1);
    }

    // Recursive function to find the majority element in a given segment
    private static int majorityRec(int[] arr, int start, int end) {
        // Base case: if the segment contains only one element
        if (start == end) {
            return arr[start];
        }

        // Find the middle index of the current segment
        int mid = start + (end - start) / 2;

        // Recursively find the majority element in the left and right halves
        int leftMajority = majorityRec(arr, start, mid);
        int rightMajority = majorityRec(arr, mid + 1, end);

        // If both halves agree on the majority element, return it
```

Run Code Result | Stdout

You must run your code first

Execute

Time and Space Complexity Analysis

- **Time Complexity**: The array is recursively divided, and at each level of recursion, counting occurrences takes O(n) time. It takes O(log n) time to divide arrays in the subarray, and the total time complexity is `O(n log n)`.
- **Space Complexity**: Due to the recursive calls, the space complexity is determined by the depth of the recursion tree, which is `O(log n)`. No additional significant space is used outside of the recursion stack.